

9-7-26 NLP SKILL TASK

```
from collections import defaultdict
```

```
class FiveGramLanguageModel:
```

```
    def __init__(self, discount=0.75):
```

```
        self.discount = discount
```

```
        self.max_order = 5
```

```
        # Sparse n-gram counts
```

```
        self.ngram_counts = [defaultdict(int) for _ in range(self.max_order)]
```

```
        self.context_counts = [defaultdict(int) for _ in range(self.max_order)]
```

```
        self.vocab = set()
```

```
        # Continuation counts for Kneser-Ney
```

```
        self.continuation_counts = defaultdict(set)
```

```
    def train(self, corpus):
```

```
        for sentence in corpus:
```

```
            words = ["<s>"] * 4 + sentence.lower().split() + ["</s>"]
```

```
            for word in words:
```

```
                self.vocab.add(word)
```

```
            length = len(words)
```

```

for n in range(1, 6):

    for i in range(length - n + 1):

        gram = tuple(words[i:i + n])

        self.ngram_counts[n - 1][gram] += 1

        if n > 1:

            context = gram[:-1]

            self.context_counts[n - 1][context] += 1

            self.continuation_counts[gram[-1]].add(context)

# Add UNK token
self.vocab.add("<UNK>")
def replace_oov(self, words):

    result = []

    for w in words:

        if w not in self.vocab:

            result.append("<UNK>")

        else:

            result.append(w)

```

```
    return result

def unigram_probability(self, word):

    continuation = len(self.continuation_counts[word])

    total = sum(len(v) for v in self.continuation_counts.values())

    if total == 0:
        return 1 / len(self.vocab)

    return continuation / total

def probability(self, ngram):

    n = len(ngram)

    if n == 1:
        return self.unigram_probability(ngram[0])

    context = ngram[:-1]

    count_ngram = self.ngram_counts[n - 1].get(ngram, 0)

    count_context = self.context_counts[n - 1].get(context, 0)

    if count_context == 0:
        return self.probability(ngram[1:])

    first = max(count_ngram - self.discount, 0) / count_context
```

```
unique_follow = 0

for gram in self.ngram_counts[n - 1]:

    if gram[:-1] == context:
        unique_follow += 1

lambda_weight = (self.discount * unique_follow) / count_context

lower = self.probability(ngram[1:])

return first + (lambda_weight * lower)
def predict(self, text, top_k=5):

    words = text.lower().split()

    words = self.replace_oov(words)

    words = ["<s>"] * 4 + words

    context = tuple(words[-4:])

    scores = []

    for word in self.vocab:

        if word == "<s>":
```

continue

gram = context + (word,)

prob = self.probability(gram)

scores.append((word, prob))

scores.sort(key=lambda x: x[1], reverse=True)

return scores[:top_k]

corpus = [

"best places to visit in india",

"best places to visit in chennai",

"best places to visit near me",

"best places to visit during summer",

"best restaurants in chennai",

"best tourist places in india",

"places to visit in kerala",

```
"places to visit in goa",

"visit india during winter",

"visit chennai beaches",

"best places to visit in tamil nadu",

"best places to visit in bangalore"
]

model = FiveGramLanguageModel()

model.train(corpus)

print("Model trained successfully!\n")

query = input("Enter your search query: ")

results = model.predict(query)

print("\nTop Search Suggestions:\n")

for word, score in results:

    print(f"{query} {word} Probability = {round(score,5)}")
```